API INTEGRATION GUIDE

Print Assistance Backend API

Complete reference for frontend integration — all endpoints, request/response schemas, auth, and the human handoff flow.

VERSION	GENERATED	TOTAL ENDPOINTS	BASE URL
1.0.0	June 28, 2026	30 APIs	/api

Table of Contents

All API groups and endpoints in this document

Send Message (JSON)	POST /api/chat/message
Send Message (SSE Stream)	POST /api/chat/stream

SECTION 5 — PRODUCTS & IMAGES

List All Products	GET /api/products
Get Product by ID	GET /api/products/:id
Get Images (Filtered)	GET /api/images

SECTION 6 — HUMAN HANDOFF (STAFF & CUSTOMER)

List Handoff Conversations	GET /api/handoff/conversations
Get Conversation Detail	GET /api/handoff/conversations/:sessionId
View via Email Token (after login)	GET /api/handoff/view/:token
Accept Conversation	POST /api/handoff/accept/:token
Assign to Another User	POST /api/handoff/assign/:token
Unassign / Re-broadcast	POST /api/handoff/unassign/:sessionId
Staff Reply to Customer	POST /api/handoff/reply/:sessionId
Close Conversation	POST /api/handoff/close/:sessionId
List Assignable Users	GET /api/handoff/assignable-users
Customer — Resume via Magic Link	GET /api/handoff/resume/:customerToken
Customer — Send Follow-up Reply	POST /api/handoff/customer-reply/:customerToken

SECTION 7 — HEALTH

Health Check	GET /health

OVERVIEW

Global Configuration

Base URL, authentication scheme, roles, and error format used across all endpoints.

BASE URL `http://localhost:5000` All paths below are appended to this base. Configure via FRONTEND_URL env.

AUTHENTICATION

JWT Bearer Token

All protected endpoints require an `Authorization: Bearer <token>` header. Tokens are issued at login, valid for 7 days. Two token types exist: Admin (hardcoded, role: "admin") and Staff/User (DB-based, role: staff | manager | viewer).

Handoff Token (Email Link)

A UUID token embedded in staff email links. Passed as a URL path param to `/api/handoff/view/:token`, `/accept/:token`, and `/assign/:token`. Valid for 7 days. *Still requires a valid JWT* — if not logged in, frontend redirects to login first.

Customer Token (Magic Link)

A UUID token for customers to resume their session without login. Embedded in the customer's email. Valid for 30 days. Used only on `/api/handoff/resume/:token` and `/api/handoff/customer-reply/:token`.

ROLES

admin	manager	staff	viewer
Full access. Hardcoded, not in User collection.	Full handoff access. Can assign & unassign.	Can accept, reply, close own conversations.	Read-only. Cannot accept, assign, reply or close.

STANDARD ERROR FORMAT

```
// All error responses follow this shape
{
```

```
  "message": "Human-readable error description"
}

// Or for validation errors:
{
  "error": "Field-level error message"
}
```

COMMON HTTP STATUS CODES

200	Success
201	Created successfully
400	Bad request — missing or invalid body fields
401	Unauthorized — missing or invalid JWT
403	Forbidden — authenticated but insufficient role
404	Resource not found
409	Conflict — duplicate resource (e.g. email already exists)
429	Rate limit exceeded
500	Internal server error

SECTION 01

Authentication

Two separate login flows: one for the hardcoded admin account, one for DB-managed staff users. Both return a JWT with a 7-day expiry.

POST `/api/admin/login` — Public

Login as the hardcoded admin account. Returns a JWT with `role: "admin"`. Admin has access to user management, full handoff control, and all menus.

REQUEST BODY

FIELD	TYPE	REQUIRED	DESCRIPTION
username	string	Required	Admin username (from env)
password	string	Required	Admin password (from env)

RESPONSE 200

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "username": "admin",
    "role": "admin"
  }
}
```

ERROR RESPONSES

401 Invalid credentials

POST /api/admin/logout

JWT — Admin only

Logout admin session. JWT is stateless — frontend should clear the stored token. Server returns a confirmation message.

RESPONSE 200

```
{ "message": "Logged out successfully" }
```

POST /api/auth/login

Public

Login for staff users (staff, manager, viewer roles). Checks DB. Inactive accounts are rejected with 403.

REQUEST BODY

FIELD	TYPE	REQUIRED	DESCRIPTION
email	string	Required	Staff email address
password	string	Required	Staff password

RESPONSE 200

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "_id": "64abc...",
    "fullName": "Jane Smith",
    "email": "jane@example.com",
    "phone": "416-555-0100",
    "role": "staff",
    "isActive": true
  }
}
```

ERROR RESPONSES

400	Missing email or password
401	Invalid credentials
403	Account is inactive

POST /api/auth/logout

JWT Required

Logout for any authenticated user. Frontend must clear the stored JWT. No body required.

RESPONSE 200

```
{ "message": "Logged out successfully" }
```

GET /api/auth/me

JWT Required

Fetch the currently logged-in user's profile. Useful on app load to restore session state. Admin returns a minimal object; staff returns full user record.

RESPONSE 200 — ADMIN

```
{ "user": { "username": "admin", "role": "admin" } }
```

RESPONSE 200 — STAFF

```
{
  "user": {
    "_id": "64abc...", "fullName": "Jane Smith",
    "email": "jane@example.com", "role": "staff",
    "isActive": true
  }
}
```

401	Missing or invalid JWT
404	User not found (shouldn't happen in normal flow)

SECTION 02

User Management

All endpoints require JWT + Admin role. Staff users are the humans who handle customer conversations.

Admin Only

Every endpoint in this section requires `role: "admin"` in the JWT. Staff, manager, and viewer tokens will receive 403.

`GET` `/api/admin/users` — JWT — Admin only

Paginated list of all staff users with optional search by name or email.

QUERY PARAMETERS

PARAM	TYPE	DEFAULT	DESCRIPTION
page	number	1	Page number
limit	number	10	Results per page
search	string	""	Search by name or email (case-insensitive)

RESPONSE 200

```
{
  "users": [
    { "_id": "...", "fullName": "Jane Smith", "email": "jane@example.com",
      "phone": "416-555-0100", "role": "staff", "isActive": true }
  ],
  "total": 24,
  "page": 1,
  "pages": 3
}
```

POST /api/admin/users

JWT — Admin only

Create a new staff user. Password is hashed server-side with bcrypt.

REQUEST BODY

FIELD	TYPE	REQUIRED	DESCRIPTION
fullName	string	Required	Full name of the user
email	string	Required	Must be unique
phone	string	Required	Contact phone number
password	string	Required	Will be bcrypt-hashed
role	string	Optional	"staff" | "manager" | "viewer" — defaults to "staff"

RESPONSE 201

```
{ "user": { "_id": "64abc...", "fullName": "...", "role": "staff", ... } }
```

409 Email already in use

GET /api/admin/users/:id

JWT — Admin only

Get a single user by MongoDB ObjectId. Returns full user object without password.

```
{ "user": { "_id": "64abc...", "fullName": "...", ... } }
```

PUT /api/admin/users/:id

JWT — Admin only

Update any field of an existing user. Only provided fields are updated. If `password` is included it will be re-hashed.

REQUEST BODY (ALL OPTIONAL)

FIELD	TYPE	DESCRIPTION
`fullName`	`string`	Updated name
`email`	`string`	Updated email
`phone`	`string`	Updated phone
`role`	`string`	"staff" | "manager" | "viewer"
`password`	`string`	New password — will be hashed

```
{ "user": { ... /* updated user object */ } }
```

PATCH /api/admin/users/:id/status

JWT — Admin only

Toggle the `isActive` flag. Active → inactive or inactive → active. No body required. Inactive users cannot log in.

```
{ "user": { "_id": "64abc...", "isActive": false, ... } }
```

DELETE /api/admin/users/:id

JWT — Admin only

Permanently delete a user. Irreversible. Consider using `PATCH /status` to deactivate instead.

```
{ "message": "User deleted successfully" }
```

SECTION 03

Dashboard & Menu

Role-aware endpoints that return different data depending on who is logged in.

GET `/api/dashboard` — JWT Required

Returns role-specific dashboard data. Admin receives aggregate stats. Staff/manager/viewer receive their own profile.

RESPONSE 200 — ADMIN

```
{
  "role": "admin",
  "stats": {
    "totalUsers": 12,
    "activeUsers": 10,
    "inactiveUsers": 2
  }
}
```

RESPONSE 200 — STAFF / MANAGER / VIEWER

```
{
  "role": "staff",
  "user": { "_id": "...", "fullName": "Jane Smith", ... }
}
```

GET /api/menu

JWT Required

Returns the navigation menu items available to the current user based on their role. Call this on app load to build the sidebar.

RESPONSE 200 — ADMIN

```
{
  "menu": [
    { "key": "users", "label": "Users", "path": "/admin/users" },
    { "key": "support", "label": "Support", "path": "/support" }
  ]
}
```

RESPONSE 200 — STAFF / MANAGER / VIEWER

```
{
  "menu": [
    { "key": "support", "label": "Support", "path": "/support" }
  ]
}
```

SECTION 04

AI Chat

Customer-facing chat endpoints. No authentication required. Rate-limited per IP.

Rate Limits

Session creation: 5 sessions per IP per hour. Messages: 10 per minute, 50 per day per IP. Message body: max 500 characters. If any limit is hit, the response is 429.

POST /api/chat/session

Public

Create a new chat session. Call this when the chat widget opens for the first time. Store the returned `sessionId` — it identifies all subsequent messages from this customer.

REQUEST BODY

No body required.

RESPONSE 201

```
{ "sessionId": "f47ac10b-58cc-4372-a567-0e02b2c3d479" }
```

429 Too many sessions created from this IP this hour

Tip: Persist `sessionId` in `localStorage` so returning visitors within the same session window don't lose history.

GET /api/chat/session/:sessionId

Public

Fetch the full session object including all messages, customer profile, and current status. Use to restore chat history on page refresh.

RESPONSE 200

```
{
  "_id": "...",
  "sessionId": "f47ac10b-...",
  "status": "active",                  // "active" | "completed" | "human_requi
  "stage": "discovery",                 // "greeting" | "discovery" | "recommen
  "messages": [
    { "role": "user", "content": "Hi I need business cards", "timestamp": "..."
    { "role": "assistant", "content": "Hey! I'm Alex...", "timestamp": "..." }
  ],
  "customerProfile": {
    "productType": "Business Cards",
    "industry": "Real Estate",
    "purpose": "Networking",
    "style": "Luxury",
    "name": "John Doe",
    "email": "john@example.com",
    "phone": "416-555-1234"
  },
  "humanReason": null,
  "userMessageCount": 5
}
```

404 Session not found

DELETE /api/chat/session/:sessionId

Public

Permanently delete a session and all its messages. Use for "start over" functionality in the chat widget.

```
{ "success": true }
```

POST `/api/chat/message` Public

Send a message to the AI and receive the full response in a single JSON response. Use this for simpler integrations. For real-time typing animation, use `/stream` instead.

REQUEST BODY

FIELD	TYPE	REQUIRED	CONSTRAINT
sessionId	string	Required	UUID from session creation
message	string	Required	Max 500 characters

RESPONSE 200


```
{
  "message": "Hey! For a real estate agent going for a luxury look, I'd recomme
  "stage": "recommending",
  "needsHuman": false,
  "humanReason": null,
  "recommendations": [
    {
      "productType": "Business Cards",
      "paperStock": "17pt Cougar Smooth",
      "finish": "Metallic Foil",
      "size": "3.5 x 2 in",
      "explanation": "Ideal for luxury real estate — the foil commands attentio
      "priceRange": "Contact us for pricing",
      "tags": ["luxury", "real-estate"]
    }
  ],
  "images": [
    { "_id": "...", "url": "https://...", "tags": { "productType": "Business Ca
  ]
}
```


When `needsHuman: true`: Stop the chat input. Show the `message` text. Display "Our team will contact you" UI. The customer's name/email/phone are now captured in their session profile and staff has been notified by email.

POST `/api/chat/stream` Public

Same as `/message` but uses Server-Sent Events (SSE) to stream the AI response token-by-token, enabling a real-time typing effect. Connect with `EventSource` or `fetch` with `ReadableStream`.

REQUEST BODY

Same as `/message` — `sessionId` and `message`.

SSE EVENT TYPES

EVENT	DATA SHAPE	WHEN
`token`	`{"chunk": "Hello"}`	Each token as the AI generates it — append to displayed text
`done`	Full response object (same as /message)	After all tokens — contains stage, needsHuman, recommendations, images
`error`	`{"error": "..."}`	If something goes wrong mid-stream
`(default)`	`[DONE]`	Stream is fully closed — safe to clean up

IMPLEMENTATION NOTES

Important: The SSE stream sends JSON tokens — do not try to parse each individual `token` event as JSON. Only parse the final `done` event data to get the structured response object. Accumulate `chunk` values and display them progressively as raw text.

SECTION 05

Products & Images

Public endpoints for browsing the product catalogue and fetching matching product images.

GET `/api/products` — Public

Returns all active products synced from WooCommerce. Used by the AI internally and optionally by the frontend for a product catalogue page.

RESPONSE 200

```
[
  {
    "_id": "64abc...",
    "name": "Business Cards",
    "minQuantity": 100,
    "active": true,
    "paperStocks": [{ "name": "14pt Cardstock" }],
    "finishes": [{ "name": "Gloss UV" }],
    "sizes": [{ "name": "Standard", "dimensions": "3.5 x 2 in" }]
  }
]
```

GET `/api/products/:id` — Public

Get a single product by its MongoDB `_id`.

```
{ /* single product object, same shape as above */ }
```

404 Product not found

GET /api/images

Public

Fetch product images filtered by tags. Used by the chat response to show visual examples alongside recommendations. All params are optional — combine any.

QUERY PARAMETERS

PARAM	TYPE	DESCRIPTION
productType	string	e.g. "Business Cards", "Flyers"
style	string	e.g. "luxury", "minimal", "bold"
industry	string	e.g. "real-estate", "restaurant"
finish	string	e.g. "Metallic Foil", "Matte"
budget	string	e.g. "low", "high"
limit	number	Max results to return (default: 10)

RESPONSE 200

```
[
  {
    "_id": "64abc...",
    "url": "https://cdn.yoursite.com/images/bc-metallic.jpg",
    "active": true,
    "tags": {
      "productType": "Business Cards",
      "style": ["luxury"],
      "industry": ["real-estate"],
      "finishes": ["Metallic Foil"]
    }
  }
]
```

SECTION 06

Human Handoff

When the AI collects a customer's name, email, and phone and sets `needsHuman: true`, the handoff system activates. Staff are emailed. The customer gets a magic link. Everything from here runs through these endpoints.

COMPLETE HANDOFF FLOW

1. **AI triggers needsHuman: true**
   chatController internally calls notifyHandoff() — creates tokens, sends emails to all staff + customer

↓

2. **Staff clicks email link → frontend checks JWT**
   Not logged in → redirect to /login?redirect=/handoff/view?token=X → after login, auto-navigate back

↓

3. **Frontend calls GET /api/handoff/view/:token**
   Server validates JWT + handoff token → returns full conversation. Chat area opens automatically.

↓

4. **Staff accepts or assigns the conversation**
   POST /accept/:token → marks as theirs. POST /assign/:token → delegates to another user.

↓

5. **Staff sends reply → customer is emailed**
   POST /reply/:sessionId → stores reply, emails customer with message + magic link

↓

6. **Customer clicks magic link → resumes chat**
   GET /resume/:customerToken → frontend restores session view. Customer sends reply via POST /customer-reply/:token

↓

7. **Staff is notified → async loop continues**

Assigned staff gets email notification. Loop repeats steps 5–7 until POST /close/:sessionId

SECTION 06 — CONTINUED

Handoff — Staff Endpoints

GET `/api/handoff/conversations` — JWT Required — All roles

List all conversations that have reached `human_required` status. Use this to build the support inbox in the dashboard. Viewer role can see but not act.

QUERY PARAMETERS

PARAM	TYPE	DEFAULT	DESCRIPTION
`status`	`string`	"all"	"unassigned" | "assigned" | "closed" | "all"
`page`	`number`	1	Page number
`limit`	`number`	10	Results per page

RESPONSE 200


```
{
  "conversations": [
    {
      "sessionId": "f47ac10b-...",
      "status": "human_required",
      "humanReason": "Customer asked about pricing",
      "customerProfile": { "name": "John Doe", "email": "john@example.com", "ph
      "assignedTo": null,
      "acceptedAt": null,
      "closedAt": null,
      "handoffNotifiedAt": "2026-07-15T10:23:00.000Z",
      "createdAt": "2026-07-15T10:20:00.000Z"
    }
  ],
  "total": 5, "page": 1, "pages": 1
}
```

GET `/api/handoff/conversations/:sessionId` JWT Required — All roles

Get the full conversation detail including all AI chat messages, staff replies, and customer replies. Used to populate the conversation view panel.

RESPONSE 200


```
{
  "sessionId": "f47ac10b-...",
  "status": "human_required",
  "humanReason": "Customer asked about pricing",
  "messages": [ /* full AI chat history */ ],
  "staffReplies": [
    { "message": "Hi John! For 500 cards, you're looking at...", "staffName": "
  ],
  "customerReplies": [
    { "message": "That sounds great, let's go ahead!", "timestamp": "..." }
  ],
  "customerProfile": { "name": "John Doe", "email": "john@...", "phone": "416-5
  "assignedTo": { "_id": "...", "fullName": "Jane Smith", "email": "jane@..." }
  "acceptedAt": "2026-07-15T10:45:00.000Z"
}
```


404 Session not found

`GET` `/api/handoff/view/:token` JWT Required + Handoff Token

The endpoint that the staff email link resolves to (after login). Validates both the handoff token *and* the JWT. If staff is not logged in when they click the email link, the frontend should redirect to `/login?redirect=/handoff/view?token=<token>` and call this after login.

> **Frontend responsibility:** On the `/handoff/view` page, check localStorage for a JWT before calling this API. If no JWT → redirect to login with `returnUrl`. After login success → redirect back and call this endpoint. The conversation chat area should open automatically on success.

RESPONSE 200

Same shape as `GET /conversations/:sessionId` above.

ERROR RESPONSES

`401`	Missing or invalid JWT
`404`	Handoff token not found or expired

POST `/api/handoff/accept/:token` — JWT Required — Staff, Manager, Admin

Staff claims this conversation for themselves. Sets `assignedTo` to the currently logged-in user. Emails all other staff who received the alert to let them know it's been claimed. **Viewer role is blocked.**

REQUEST BODY

No body required. User identity comes from JWT.

RESPONSE 200

```
{
  "success": true,
  "assignedTo": { "_id": "...", "fullName": "Jane Smith", "email": "jane@..." }
  "acceptedAt": "2026-07-15T10:45:00.000Z"
}
```

400	Already assigned to someone — use /assign to reassign
403	Viewer role cannot accept conversations
404	Handoff token not found or expired

POST `/api/handoff/assign/:token` — JWT Required — Manager, Admin only

Manager or admin assigns the conversation to a specific staff member. The assigned staff receives a dedicated email with a handoff link. Overwrites any existing assignment.

REQUEST BODY

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	`string`	Required	MongoDB ObjectId of the user to assign to

RESPONSE 200

```
{
  "success": true,
  "assignedTo": { "_id": "...", "fullName": "Bob Jones", "email": "bob@..." }
}
```

403	Staff and viewer roles cannot assign — only manager or admin
404	Handoff token or target userId not found

SECTION 06 — CONTINUED

Handoff — Staff Actions & Customer Endpoints

POST `/api/handoff/unassign/:sessionId` JWT Required — Manager, Admin only

Release the current assignment from a conversation. All original staff are re-notified by email so anyone can pick it up again. Use when the assigned person is unavailable or the assignment was incorrect.

RESPONSE 200

```
{ "success": true, "message": "Conversation unassigned and staff re-notified" }
```

400	Conversation is not currently assigned
403	Only manager or admin can unassign

POST /api/handoff/reply/:sessionId

JWT Required — Staff, Manager, Admin

Send a reply to the customer. Stores the message in `session.staffReplies` and triggers an email to the customer containing the reply text and their magic link to respond. The email appears to come from the staff member's name.

REQUEST BODY

FIELD	TYPE	REQUIRED	DESCRIPTION
message	string	Required	The reply message to send to the customer

RESPONSE 200

```
{ "success": true, "message": "Reply sent and customer notified by email" }
```

- **400** Message body is required
- **403** Viewer role cannot reply
- **404** Session not found

POST /api/handoff/close/:sessionId

JWT Required — Staff, Manager, Admin

Mark a conversation as resolved. Sets `session.status = "completed"` and stamps `closedAt`. Once closed, the customer's magic link still works to view history but the customer-reply endpoint will inform them the conversation is closed.

RESPONSE 200

```
{ "success": true, "closedAt": "2026-07-15T14:30:00.000Z" }
```

- **400** Conversation is already closed
- **403** Viewer role cannot close conversations

GET /api/handoff/assignable-users

JWT Required — All roles

Returns a list of active users (staff, manager) who can be assigned a conversation. Use this to populate the "Assign to" dropdown on the handoff UI.

RESPONSE 200


```
{
  "users": [
    { "_id": "64abc...", "fullName": "Jane Smith", "email": "jane@...", "role":
    { "_id": "64def...", "fullName": "Bob Jones", "email": "bob@...", "role": "
  ]
}
```

CUSTOMER — TOKEN-BASED (NO LOGIN REQUIRED)

`GET` `/api/handoff/resume/:customerToken` Customer Token

Called when the customer clicks their magic link from the email. No login needed. Validates the customer token and returns the full session (AI messages + all staff replies + their own replies) so the frontend can re-render the conversation view.

> **No login required**
> The `customerToken` in the URL path is the only credential needed. Extract it from the magic link URL and pass it here. Token is valid for 30 days.

RESPONSE 200

```
{
  "sessionId": "f47ac10b-...",
  "status": "human_required",
  "customerProfile": { "name": "John Doe", "email": "john@...", ... },
  "messages": [ /* original AI chat messages */ ],
  "staffReplies": [
    { "message": "Hi John! Here's the pricing...", "staffName": "Jane Smith", "
  ],
  "customerReplies": [ /* customer's previous replies */ ],
  "customerToken": "3c0e5d9b-..."  // pass this back to /customer-reply
}
```

`404` Customer token not found or expired (30 days)

POST `/api/handoff/customer-reply/:customerToken` — Customer Token

Customer sends a follow-up reply via the magic link conversation view. Stores in `session.customerReplies` and emails the assigned staff member (or all staff if unassigned) that the customer has responded.

REQUEST BODY

FIELD	TYPE	REQUIRED	DESCRIPTION
message	string	Required	The customer's reply message

RESPONSE 200


```
{ "success": true, "message": "Reply sent — a team member will follow up shortl
```


400	Message is required | Conversation is already closed
404	Customer token not found or expired

SECTION 07

Health Check

GET /health — Public

Simple liveness check. Returns 200 if the server is running. No auth required. Use in load balancers or uptime monitors.

```
{ "status": "ok" }
```

Environment Variables

Configure before running the server.

VARIABLE	REQUIRED	DESCRIPTION
MONGODB_URI	Required	MongoDB connection string
JWT_SECRET	Required	Secret for signing JWTs. Must be set in production.
OPENAI_API_KEY	Required	OpenAI API key for AI chat
ADMIN_USERNAME	Required	Username for the admin login
ADMIN_PASSWORD	Required	Password for the admin login
EMAIL_HOST	Required	SMTP host (e.g. smtp.gmail.com)
EMAIL_PORT	Required	SMTP port (587 for TLS, 465 for SSL)
EMAIL_USER	Required	SMTP login email address
EMAIL_PASS	Required	SMTP password or app-specific password
EMAIL_FROM	Required	Display name + address for outgoing email (e.g. "PrintAssistance <no-reply@printco.com>")

FRONTEND_URL	Optional	Frontend base URL for CORS and magic links (default: http://localhost:3000)
OPENAI_MODEL	Optional	OpenAI model to use (default: gpt-4o-mini)
PORT	Optional	Server port (default: 5000)

Quick Reference — All Endpoints

METHOD	ENDPOINT	AUTH	PURPOSE
POST	/api/admin/login	Public	Admin login
POST	/api/admin/logout	Admin JWT	Admin logout
POST	/api/auth/login	Public	Staff login
POST	/api/auth/logout	JWT	Staff logout
GET	/api/auth/me	JWT	Current user profile
GET	/api/admin/users	Admin JWT	List users (paginated)
POST	/api/admin/users	Admin JWT	Create user
GET	/api/admin/users/:id	Admin JWT	Get user
PUT	/api/admin/users/:id	Admin JWT	Update user
PATCH	/api/admin/users/:id/status	Admin JWT	Toggle active status
DELETE	/api/admin/users/:id	Admin JWT	Delete user
GET	/api/dashboard	JWT	Dashboard stats / profile
GET	/api/menu	JWT	Role-based nav menu
POST	/api/chat/session	Public	Create chat session
GET	/api/chat/session/:sessionId	Public	Get session + messages
DELETE	/api/chat/session/:sessionId	Public	Delete session

POST	/api/chat/message	Public	Send message (JSON)
POST	/api/chat/stream	Public	Send message (SSE stream)
GET	/api/products	Public	List products
GET	/api/products/:id	Public	Get product
GET	/api/images	Public	Get filtered images
GET	/api/handoff/conversations	JWT (all roles)	List handoff conversations
GET	/api/handoff/conversations/:sessionId	JWT (all roles)	Conversation detail
GET	/api/handoff/view/:token	JWT + Handoff Token	View via email link
POST	/api/handoff/accept/:token	JWT (staff+)	Accept conversation
POST	/api/handoff/assign/:token	JWT (manager+)	Assign to user
POST	/api/handoff/unassign/:sessionId	JWT (manager+)	Release assignment
POST	/api/handoff/reply/:sessionId	JWT (staff+)	Staff reply to customer
POST	/api/handoff/close/:sessionId	JWT (staff+)	Close conversation
GET	/api/handoff/assignable-users	JWT (all roles)	Users for assign dropdown
GET	/api/handoff/resume/:customerToken	Customer Token	Customer resumes via magic link
POST	/api/handoff/customer-reply/:customerToken	Customer Token	Customer sends follow-up
GET	/health	Public	Server health check